MCA Lab - 1

Name : Abhay Kiran M

Reg No : 261100620002

Board : STM32F103C8T6 (Blue pill), Keil uVision 5, ST-Link V2

SET 1

1. While choosing the board, what information we understand?

Before using the board we checked the datasheet and the schematic of the blue pill board. The controller is STM32F103C8T6 which is ARM Cortex-M3 based, 32 bit, and can run upto 72 MHz using the 8 MHz external crystal and PLL.

It has 64 KB flash and 20 KB SRAM. Operating voltage is 3.3 V (2 to 3.6 V), some pins are 5 V tolerant. It has 37 GPIO pins in port A, B, C and D. Peripherals available are 3 USART, 2 SPI, 2 I2C, CAN, USB, 2 ADC of 12 bit, timers TIM1 to TIM4, SysTick and watchdog.

On the board there is a power LED, one user LED on PC13, reset switch, BOOT0 and BOOT1 jumpers and micro USB. Programming is done using SWD (PA13, PA14) with ST-Link.

From this we came to know how much memory we have, which pins are free to connect LED and switch, which pins should not be used (PA13/PA14 for debug, PC14/PC15 for 32 kHz crystal, PA11/PA12 for USB), voltage level for interfacing and maximum clock frequency.

2. What do you mean by startup.s or significance of it?

startup.s is an assembly file which runs first after the controller is reset, before main(). For our board Keil adds startup_stm32f10x_md.s (md means medium density device).

It reserves stack and heap, has the vector table which is placed at the starting of flash, and has the Reset_Handler. Reset_Handler calls SystemInit() for clock setting and then goes to __main, which initialises the variables and calls our main() function. It also has default handlers for all the interrupts and exceptions.

3. What is the necessity of startup.s code?

After reset nothing is ready for running C code. The startup code is needed because:

1. Stack pointer has to be loaded. The CPU takes the first value of the vector table as SP. Without stack, function calls and local variables won't work.
2. CPU needs the reset vector to know from where to start executing.
3. Initialised global variables have to be copied from flash to RAM and uninitialised ones have to be made zero (done by __main which is called from startup).
4. SystemInit has to be called so that the controller runs at 72 MHz, otherwise it runs at 8 MHz HSI.
5. When an interrupt or fault comes, CPU should know which handler to jump to, that is given by the vector table.

4. Analyse a startup code and write a brief summary about it.

File: startup_stm32f10x_md.s (from Keil project, opened in vim)

```
 33 Stack_Size      EQU     0x00000400
 34
 35                 AREA    STACK, NOINIT, READWRITE, ALIGN=3
 36 Stack_Mem       SPACE   Stack_Size
 37 __initial_sp
 38
 39
 40 ; <h> Heap Configuration
 41 ;   <o>  Heap Size (in Bytes) <0x0-0xFFFFFFFF:8>
 42 ; </h>
 43
 44 Heap_Size       EQU     0x00000200
 45
 46                 AREA    HEAP, NOINIT, READWRITE, ALIGN=3
 47 __heap_base
 48 Heap_Mem        SPACE   Heap_Size
 49 __heap_limit
 50
 51                 PRESERVE8
 52                 THUMB
 53
 54
 55 ; Vector Table Mapped to Address 0 at Reset
 56                 AREA    RESET, DATA, READONLY
 57                 EXPORT  __Vectors
 58                 EXPORT  __Vectors_End
 59                 EXPORT  __Vectors_Size
```

```
 60
 61 __Vectors       DCD     __initial_sp               ; Top of Stack
 62                 DCD     Reset_Handler              ; Reset Handler
 63                 DCD     NMI_Handler                ; NMI Handler
 64                 DCD     HardFault_Handler          ; Hard Fault Handler
 65                 DCD     MemManage_Handler          ; MPU Fault Handler
 66                 DCD     BusFault_Handler           ; Bus Fault Handler
 67                 DCD     UsageFault_Handler         ; Usage Fault Handler
 68                 DCD     0                          ; Reserved
 69                 DCD     0                          ; Reserved
 70                 DCD     0                          ; Reserved
 71                 DCD     0                          ; Reserved
 72                 DCD     SVC_Handler                ; SVCall Handler
 73                 DCD     DebugMon_Handler           ; Debug Monitor
Handler
 74                 DCD     0                          ; Reserved
 75                 DCD     PendSV_Handler             ; PendSV Handler
 76                 DCD     SysTick_Handler            ; SysTick Handler
 77
 78                 ; External Interrupts
 79                 DCD     WWDG_IRQHandler            ; Window Watchdog
 80                 DCD     PVD_IRQHandler             ; PVD through EXTI
Line detect
 81                 DCD     TAMPER_IRQHandler          ; Tamper
```

```
140                 AREA    |.text|, CODE, READONLY
141
142 ; Reset handler
143 Reset_Handler    PROC
144                  EXPORT  Reset_Handler             [WEAK]
145      IMPORT  __main
146      IMPORT  SystemInit
147                  LDR     R0, =SystemInit
148                  BLX     R0
149                  LDR     R0, =__main
150                  BX      R0
151                  ENDP
152
153 ; Dummy Exception Handlers (infinite loops which can be modified)
154
155 NMI_Handler     PROC
156                 EXPORT  NMI_Handler               [WEAK]
157                 B       .
```

```
158                 ENDP
159 HardFault_Handler\
160                 PROC
161                 EXPORT  HardFault_Handler          [WEAK]
162                 B       .
163                 ENDP
```

Summary:

1. Stack size is 0x400 (1 KB) and heap size is 0x200 (512 bytes). __initial_sp is the top of stack.
2. PRESERVE8 keeps the stack 8 byte aligned and THUMB means thumb instructions are used, since Cortex-M3 supports only thumb-2.
3. RESET area is the vector table, it goes to address 0x08000000. First word is initial SP value, second is address of Reset_Handler, then the system exceptions and after that the external interrupts (60 for md devices). Reserved places are 0.
4. Reset_Handler first calls SystemInit (in system_stm32f10x.c) for clock setup and then branches to __main. __main is from C library, it copies .data to RAM, clears .bss and then calls main().
5. All other handlers are just "B ." which is an infinite loop. They are declared [WEAK] so if we write a function with the same name in our C file (like SysTick_Handler) ours will be used.
6. At the end of the file __user_initial_stackheap gives the stack and heap addresses to the C library (if MicroLIB is not used).

5. What is the significance of the definition given, clock setups, entry points, vector addresses, exceptions, reset handlers, etc?

Stack and heap definition: decides how much RAM is used for stack (function calls, local variables) and heap (malloc). If the stack is less the program can go to HardFault.

Vector addresses: vector table is at the start of flash 0x08000000. When BOOT0 = 0 the flash is aliased to 0x00000000, so on reset CPU reads SP from 0x00000000 and reset handler address from 0x00000004. Next ones are NMI at 0x08, HardFault at 0x0C, MemManage 0x10, BusFault 0x14, UsageFault 0x18, SVCall 0x2C, PendSV 0x38, SysTick 0x3C. Peripheral interrupts start from 0x40 (WWDG is IRQ0), for example EXTI0 is at 0x58 and USART1 at 0xD4.

Exceptions: exception numbers 1 to 15 are of the Cortex-M3 core like reset, NMI, faults, SVC, PendSV, SysTick. Faults come when something goes wrong, like accessing a wrong address. From 16 onwards are the peripheral interrupts handled by NVIC.

Reset handler: first code to execute after power on or reset. It does clock setting through SystemInit and then gives control to C startup code.

Clock setup: after reset the controller works on HSI 8 MHz. SystemInit turns on HSE (8 MHz crystal), sets 2 wait states for flash, PLL multiplier 9, and selects PLL as system clock. So SYSCLK = 72 MHz, APB2 = 72 MHz, APB1 = 36 MHz. Our delay and baud rate values are calculated based on this.

Entry point: the actual entry point is Reset_Handler, not main. Flow is Reset_Handler, SystemInit, __main, main().

WEAK: lets us override the default handler by writing our own function with the same name.

Boot pins: BOOT0 = 0 boots from flash, BOOT0 = 1 and BOOT1 = 0 boots from system memory (bootloader), both 1 boots from SRAM.

SET 2

1. Check built-in LED connection, identify the layout, pins

From the blue pill schematic, the user LED (green) is connected to PC13. Anode of the LED is connected to 3.3 V through a resistor and cathode goes to PC13. So the LED is active low, when PC13 is 0 LED glows and when PC13 is 1 it is off. The red LED is power LED, it is directly on 3.3 V so we cannot control it.

PC13 can sink only around 3 mA so it is used only for the LED, and output speed is kept at 2 MHz.

Connections used for this lab:

PC13 - on board LED (active low)

PA1, PA2, PA3, PA4 - external LEDs with 330 ohm resistor to GND

PB12 - push button to GND (internal pull up used)

PA9 (TX), PA10 (RX) - USB to TTL converter for serial output to PC

PA13, PA14 - SWD to ST-Link

All the programs print what they are doing on USART1 at 9600 baud and the output was seen on PuTTY (COM5).

Program to test PC13 LED:

```
 1 #include "lab1_002.h"
 2
 3 // test for on board led PC13
 4 int count = 0;
 5
 6 int main(void){
 7     gpio_init_002();
 8     uart_init_002();
 9     uart_print_002("PC13 board led test\r\n");
10
11     while(1){
12         GPIOC->BRR = (1<<13);   // write 0
13         uart_print_002("PC13 = 0 -> LED ON\r\n");
14         delay_ms_002(1000);
15
16         GPIOC->BSRR = (1<<13); // write 1
17         uart_print_002("PC13 = 1 -> LED OFF\r\n");
18         delay_ms_002(1000);
19
20         count++;
21         if(count == 3){
22             uart_print_002("led is active low\r\n");
23         }
24     }
25 }
~
~
"main.c" 25L, 545B
```

Build and download:

```
Build started: Project: lab1
*** Using Compiler 'V5.06 update 7 (build 960)', folder:
'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Target 1'
compiling main.c...
compiling lab1_002.c...
linking...
Program Size: Code=1020 RO-data=300 RW-data=4 ZI-data=1540
".\Objects\lab1.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:01
Load "C:\\Users\\LAB-PC\\Desktop\\abhay\\mca_lab1\\Objects\\lab1.axf"
Erase Done.
Programming Done.
Verify OK.
Application running ...
Flash Load finished at 10:07:41
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 10:08:02 =~=~=~=~=~=~=~=~=~=~=~=
PC13 board led test
PC13 = 0 -> LED ON
PC13 = 1 -> LED OFF
PC13 = 0 -> LED ON
PC13 = 1 -> LED OFF
PC13 = 0 -> LED ON
PC13 = 1 -> LED OFF
led is active low
PC13 = 0 -> LED ON
PC13 = 1 -> LED OFF
PC13 = 0 -> LED ON
```

LED glowed when 0 was written to PC13 and turned off when 1 was written, so it is active low.

2. Understand the header file

stm32f10x.h has structures for all peripherals and their base addresses, so we can access registers like GPIOA->ODR instead of writing the full address. __IO is volatile.

```c
/* stm32f10x.h  (only the parts used in this lab) */

typedef struct
{
  __IO uint32_t CRL;
  __IO uint32_t CRH;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
  __IO uint32_t BRR;
  __IO uint32_t LCKR;
} GPIO_TypeDef;

typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t CFGR;
  __IO uint32_t CIR;
  __IO uint32_t APB2RSTR;
  __IO uint32_t APB1RSTR;
  __IO uint32_t AHBENR;
  __IO uint32_t APB2ENR;
  __IO uint32_t APB1ENR;
  __IO uint32_t BDCR;
  __IO uint32_t CSR;
} RCC_TypeDef;

#define FLASH_BASE            ((uint32_t)0x08000000)
#define SRAM_BASE             ((uint32_t)0x20000000)
#define PERIPH_BASE           ((uint32_t)0x40000000)

#define APB1PERIPH_BASE       PERIPH_BASE
#define APB2PERIPH_BASE       (PERIPH_BASE + 0x10000)
#define AHBPERIPH_BASE        (PERIPH_BASE + 0x20000)

#define AFIO_BASE             (APB2PERIPH_BASE + 0x0000)
#define GPIOA_BASE            (APB2PERIPH_BASE + 0x0800)
#define GPIOB_BASE            (APB2PERIPH_BASE + 0x0C00)
#define GPIOC_BASE            (APB2PERIPH_BASE + 0x1000)
#define USART1_BASE           (APB2PERIPH_BASE + 0x3800)
#define RCC_BASE              (AHBPERIPH_BASE + 0x1000)

#define GPIOA                ((GPIO_TypeDef *) GPIOA_BASE)
```

```
 44 #define GPIOB               ((GPIO_TypeDef *) GPIOB_BASE)
 45 #define GPIOC               ((GPIO_TypeDef *) GPIOC_BASE)
 46 #define RCC                 ((RCC_TypeDef *) RCC_BASE)
 47 #define USART1              ((USART_TypeDef *) USART1_BASE)
 48
 49 #define  RCC_APB2ENR_AFIOEN                  ((uint32_t)0x00000001)
 50 #define  RCC_APB2ENR_IOPAEN                  ((uint32_t)0x00000004)
 51 #define  RCC_APB2ENR_IOPBEN                  ((uint32_t)0x00000008)
 52 #define  RCC_APB2ENR_IOPCEN                  ((uint32_t)0x00000010)
 53 #define  RCC_APB2ENR_USART1EN                ((uint32_t)0x00004000)
~
~
"stm32f10x.h" [readonly] 53L, 1785B
```

Register description (GPIO): every port has 7 registers.

CRL (offset 0x00) - configuration of pin 0 to 7, 4 bits for each pin

CRH (offset 0x04) - configuration of pin 8 to 15

IDR (offset 0x08) - input data, used to read button

ODR (offset 0x0C) - output data, in input mode it selects pull up (1) or pull down (0)

BSRR (offset 0x10) - lower 16 bits set the pin, upper 16 bits reset the pin

BRR (offset 0x14) - reset the pin

LCKR (offset 0x18) - lock configuration

4 bits of each pin are CNF1 CNF0 MODE1 MODE0. Values used:

0x2 - output push pull 2 MHz (LED)

0x8 - input pull up / pull down (button)

0x4 - floating input (PA10 RX, this is also the reset value)

0xB - alternate function push pull 50 MHz (PA9 TX)

For PC13 the bits are 20 to 23 in CRH, so we write 0x2<<20. For PB12 the bits are 16 to 19 in CRH, so 0x8<<16.

Clock for GPIO and USART1 must be enabled in RCC->APB2ENR first. Bit 0 AFIO, bit 2 port A, bit 3 port B, bit 4 port C, bit 14 USART1.

Address mapping:

Peripheral base = 0x40000000

APB2 base = 0x40010000, AHB base = 0x40020000

GPIOA = 0x40010800, GPIOB = 0x40010C00, GPIOC = 0x40011000

USART1 = 0x40013800

RCC = 0x40021000

SysTick = 0xE000E010, NVIC = 0xE000E100

Example: GPIOC->ODR = 0x40011000 + 0x0C = 0x4001100C

RCC->APB2ENR = 0x40021000 + 0x18 = 0x40021018

Memory locations:

Flash : 0x08000000 to 0x0800FFFF (64 KB)

SRAM : 0x20000000 to 0x20004FFF (20 KB)

System memory (bootloader) : 0x1FFFF000 to 0x1FFFF7FF

Option bytes : 0x1FFFF800

Peripherals : from 0x40000000

Cortex-M3 internal peripherals : 0xE0000000 to 0xE00FFFFF

In Keil target options IROM1 is 0x8000000 size 0x10000 and IRAM1 is 0x20000000 size 0x5000.

3. WAP

Common files used for all programs (only main.c is changed for each question):

lab1_002.h

```
 1 /* common functions for mca lab 1 */
 2
 3 #ifndef LAB1_002_H
 4 #define LAB1_002_H
 5
 6 #include "stm32f10x.h"
 7
 8 // pins used
 9 #define LED_PC13  13  // on board led (active low)
10 #define LED1      1 // PA1
11 #define LED2      2 // PA2
12 #define LED3      3   // PA3
```

```
 13 #define LED4      4   // PA4
 14 #define BTN       12 // PB12 (internal pull up, pressed = 0)
 15
 16 void gpio_init_002(void);
 17 void delay_ms_002(unsigned int ms);
 18
 19 void led_on_002(int pin);
 20 void led_off_002(int pin);
 21 void board_led_on_002(void);
 22 void board_led_off_002(void);
 23 int  button_read_002(void);
 24
 25 void uart_init_002(void);
 26 void uart_char_002(char c);
 27 void uart_print_002(char *s);
 28 void uart_num_002(unsigned int n);
 29
 30 #endif
~
~
"lab1_002.h" 30L, 670B
```

lab1_002.c

```
  1 #include "lab1_002.h"
  2
  3 // clocks + pin modes for led and button
  4 void gpio_init_002(void){
  5     RCC->APB2ENR |= (1<<2) | (1<<3) | (1<<4);  // GPIOA, GPIOB, GPIOC
clock
  6
  7     // PC13 -> output push pull 2MHz
  8     GPIOC->CRH &= ~(0xF<<20);
  9     GPIOC->CRH |=  (0x2<<20);
 10     GPIOC->BSRR = (1<<13);  // led off (active low)
 11
 12     // PA1 to PA4 -> output push pull 2MHz
 13     GPIOA->CRL &= ~(0xFFFF<<4);
 14     GPIOA->CRL |=  (0x2222<<4);
 15     GPIOA->BRR = (0xF<<1);   // all off
 16
 17     // PB12 -> input with pull up
 18     GPIOB->CRH &= ~(0xF<<16);
```

```
19      GPIOB->CRH |=  (0x8<<16);
20      GPIOB->ODR |=  (1<<12); // select pull up
21  }
22
23  //void delay(int d){ while(d--); }   // old delay, not accurate
24
25  // systick delay, 72MHz -> 72000 counts = 1ms
26  void delay_ms_002(unsigned int ms){
27      unsigned int i;
28      SysTick->LOAD = 72000 - 1;
29      SysTick->VAL = 0;
30      SysTick->CTRL = 5; // enable, cpu clock, no int
31      for(i = 0; i < ms; i++){
32          while((SysTick->CTRL & (1<<16)) == 0);  // wait countflag
33      }
34      SysTick->CTRL = 0;
35  }
36
37  void led_on_002(int pin){
38      GPIOA->BSRR = (1<<pin);
39  }
40
41  void led_off_002(int pin){
42      GPIOA->BRR = (1<<pin);
43  }
44
45  void board_led_on_002(void){
46      GPIOC->BRR = (1<<13);    // low = on
47  }
48
49  void board_led_off_002(void){
50      GPIOC->BSRR = (1<<13);
51  }
52
53  // returns 1 when pressed
54  int button_read_002(void){
55      if((GPIOB->IDR & (1<<BTN)) == 0){
56          return 1;
57      }
58      return 0;
59  }
60
61  // USART1 PA9 TX, PA10 RX, 9600 baud
```

```
 62 void uart_init_002(void){
 63     RCC->APB2ENR |= (1<<14) | (1<<2) | (1<<0);  // USART1, GPIOA, AFIO
 64
 65     GPIOA->CRH &= ~(0xFF<<4);
 66     GPIOA->CRH |=  (0xB<<4); // PA9 AF push pull 50MHz
 67     GPIOA->CRH |=  (0x4<<8);  // PA10 input floating
 68
 69     USART1->BRR = 0x1D4C;    // 72MHz / 9600 = 468.75
 70     USART1->CR1 = (1<<13) | (1<<3) | (1<<2); // UE, TE, RE
 71 }
 72
 73 void uart_char_002(char c){
 74     while((USART1->SR & (1<<7)) == 0);    // wait TXE
 75     USART1->DR = c;
 76 }
 77
 78 void uart_print_002(char *s){
 79     while(*s){
 80         uart_char_002(*s);
 81         s++;
 82     }
 83 }
 84
 85 void uart_num_002(unsigned int n){
 86     char buf[10];
 87     int i = 0;
 88     if(n == 0){
 89         uart_char_002('0');
 90         return;
 91     }
 92     while(n > 0){
 93         buf[i] = (n % 10) + '0';
 94         n = n / 10;
 95         i++;
 96     }
 97     while(i > 0){
 98         i--;
 99         uart_char_002(buf[i]);
100     }
101 }
~
~
"lab1_002.c" 101L, 2269B written
```

Baud rate: 72 MHz / (16 x 9600) = 468.75, mantissa 468 = 0x1D4, fraction 0.75 x 16 = 12 = 0xC, so BRR = 0x1D4C.

Delay: SysTick LOAD = 72000 - 1 gives 1 ms at 72 MHz.

3.1 Connect LED and blink

Aim: To blink LED connected to PA1 with 500 ms delay.

```
  1 // Q3.1 led blink
  2 // Abhay Kiran M  261100620002
  3 #include "lab1_002.h"
  4
  5 unsigned int t = 0;    // time in ms (counted by us)
  6 int blink_delay = 500;
  7
  8 int main(void){
  9     gpio_init_002();
 10     uart_init_002();
 11     uart_print_002("Q3.1 LED blink on PA1\r\n");
 12
 13     while(1){
 14         //GPIOA->ODR ^= (1<<1);   // tried toggle first
 15         led_on_002(LED1);
 16         uart_print_002("t=");
 17         uart_num_002(t);
 18         uart_print_002(" ms  LED ON\r\n");
 19         delay_ms_002(blink_delay);
 20         t = t + blink_delay;
 21
 22         led_off_002(LED1);
 23         uart_print_002("t=");
 24         uart_num_002(t);
 25         uart_print_002(" ms  LED OFF\r\n");
 26         delay_ms_002(blink_delay);
 27         t = t + blink_delay;
 28     }
 29 }
~
~
"main.c" 29L, 714B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 10:21:15 =~=~=~=~=~=~=~=~=~=~=
ÿQ3.1 LED blink on PA1
t=0 ms  LED ON
t=500 ms  LED OFF
t=1000 ms  LED ON
t=1500 ms  LED OFF
t=2000 ms  LED ON
t=2500 ms  LED OFF
t=3000 ms  LED ON
t=3500 ms  LED OFF
t=4000 ms  LED ON
t=4500 ms  LED OFF
t=5000 ms  LED ON
t=5500 ms  LED OFF
t=6000 ms  LED ON
t=6500 ms  LED OFF
t=7000 ms  LED ON
t=7500 ms  LED OFF
t=80
```

LED blinked with 500 ms ON and 500 ms OFF.

3.2 Connect multiple LEDs and glow with desired delay

Aim: To connect 4 LEDs (PA1 to PA4) and glow them one by one with the delay given in my_delay.

```
 1 #include "lab1_002.h"
 2
 3 // 3.2 four leds PA1-PA4, glow one after other
 4 int led_pin[4] = {LED1, LED2, LED3, LED4};
 5 int my_delay = 300;  //1000   // change this for desired delay
 6 int i;
 7
 8 int main(void){
 9     gpio_init_002();
10     uart_init_002();
11     uart_print_002("Q3.2 multiple LEDs, delay = ");
12     uart_num_002(my_delay);
13     uart_print_002(" ms\r\n");
14
15     while(1){
16         // one by one
```

```
 17         for(i = 0; i < 4; i++){
 18             led_on_002(led_pin[i]);
 19             uart_print_002("LED");
 20             uart_num_002(i + 1);
 21             uart_print_002(" ON\r\n");
 22             delay_ms_002(my_delay);
 23             led_off_002(led_pin[i]);
 24         }
 25
 26         // all on
 27         for(i = 0; i < 4; i++){
 28             led_on_002(led_pin[i]);
 29         }
 30         uart_print_002("ALL ON\r\n");
 31         delay_ms_002(my_delay * 2);
 32
 33         // all off
 34         for(i = 0; i < 4; i++){
 35             led_off_002(led_pin[i]);
 36         }
 37         uart_print_002("ALL OFF\r\n");
 38         delay_ms_002(my_delay * 2);
 39     }
 40 }
~
~
"main.c" 40L, 1001B
```

Output (my_delay = 300):

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 10:34:50 =~=~=~=~=~=~=~=~=~=~=~=
Q3.2 multiple LEDs, delay = 300 ms
LED1 ON
LED2 ON
LED3 ON
LED4 ON
ALL ON
ALL OFF
LED1 ON
LED2 ON
LED3 ON
LED4 ON
ALL ON
ALL OFF
LED1 ON
LED2 ON
LED3 ON
```

Output (my_delay = 1000):

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 10:37:26 =~=~=~=~=~=~=~=~=~=~=~=
Q3.2 multiple LEDs, delay = 1000 ms
LED1 ON
LED2 ON
LED3 ON
LED4 ON
ALL ON
ALL OFF
LED1 ON
LED2 ON
```

LEDs glowed one after another and then all together. With 1000 ms the sequence was slower.

3.3 Connect LED and button, if button is pressed LED must glow

Aim: LED on PA1 should glow as long as the button on PB12 is pressed.

```
1 #include "lab1_002.h"
2
3 // 3.3 led glows while button is pressed
4 int btn = 0;
5 int last_btn = 0;
```

```
  6
  7 int main(void){
  8     gpio_init_002();
  9     uart_init_002();
 10     uart_print_002("Q3.3 LED on while button pressed\r\n");
 11
 12     while(1){
 13         btn = button_read_002();
 14
 15         if(btn == 1){
 16             led_on_002(LED1);
 17         }
 18         else{
 19             led_off_002(LED1);
 20         }
 21
 22         // print only when state changes
 23         if(btn != last_btn){
 24             if(btn == 1){
 25                 uart_print_002("Button PRESSED  -> LED ON\r\n");
 26             }
 27             else{
 28                 uart_print_002("Button RELEASED -> LED OFF\r\n");
 29             }
 30             last_btn = btn;
 31         }
 32         delay_ms_002(10); // small debounce
 33     }
 34 }
~
~
"main.c" 34L, 744B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 10:46:09 =~=~=~=~=~=~=~=~=~=~=~=
Q3.3 LED on while button pressed
Button PRESSED  -> LED ON
Button RELEASED -> LED OFF
Button PRESSED  -> LED ON
Button RELEASED -> LED OFF
Button PRESSED  -> LED ON
Button RELEASED -> LED OFF
Button PRESSED  -> LED ON
Button RELEASED -> LED OFF
Button PRESSED  -> LED ON
Button RELEASED -> LED OFF
Button PRESSED  -> LED ON
Button RELEASED -> LED OFF
```

LED was ON only while the button was pressed. We pressed the button 5 times but got 6 lines, one extra came because of switch bounce.

3.4 Modify 3rd question, LED must glow after releasing the button

Aim: LED should not glow while the button is held, it should glow only after releasing the button.

```
 1 #include "lab1_002.h"
 2
 3 // 3.4 led glows only after button is released
 4 int glow_time = 2000;    // led on time after release
 5 int count = 0;
 6
 7 int main(void){
 8     gpio_init_002();
 9     uart_init_002();
10     uart_print_002("Q3.4 LED on after button release\r\n");
11
12     while(1){
13         if(button_read_002() == 1){
14             delay_ms_002(20); // debounce
15             if(button_read_002() == 1){
16                 uart_print_002("Button pressed, LED still OFF\r\n");
17                 while(button_read_002() == 1);  // wait for release
18                 delay_ms_002(20);
19
20                 count++;
```

```
 21                 uart_print_002("Button released -> LED ON  (");
 22                 uart_num_002(count);
 23                 uart_print_002(")\r\n");
 24                 led_on_002(LED1);
 25                 delay_ms_002(glow_time);
 26                 led_off_002(LED1);
 27                 uart_print_002("LED OFF after ");
 28                 uart_num_002(glow_time);
 29                 uart_print_002(" ms\r\n");
 30             }
 31         }
 32     }
 33 }
~
~
"main.c" 33L, 1006B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 11:02:33 =~=~=~=~=~=~=~=~=~=~=~=
Q3.4 LED on after button release
Button pressed, LED still OFF
Button released -> LED ON  (1)
LED OFF after 2000 ms
Button pressed, LED still OFF
Button released -> LED ON  (2)
LED OFF after 2000 ms
Button pressed, LED still OFF
Button released -> LED ON  (3)
LED OFF after 2000 ms
```

While holding the button LED was off, after releasing it glowed for 2 sec.

3.5 Button pressed once LED glow once, pressed twice LED glow twice

Aim: Count how many times the button is pressed and blink the LED that many times. After a press the program waits 1 sec for the next press, if no press comes it blinks the LED.

```
  1 #include "lab1_002.h"
  2
  3 // 3.5 led blinks as many times as button pressed
  4 int press_count = 0;
```

```
int wait_time = 0;
int i;

// waits till button released, used for counting
void wait_release_002(void){
    delay_ms_002(20);
    while(button_read_002() == 1);
    delay_ms_002(20);
}

int main(void){
    gpio_init_002();
    uart_init_002();
    uart_print_002("Q3.5 LED glows count = button press count\r\n");

    while(1){
        if(button_read_002() == 1){
            wait_release_002();
            press_count = 1;
            wait_time = 0;

            // wait 1 sec for more presses
            while(wait_time < 1000){
                if(button_read_002() == 1){
                    wait_release_002();
                    press_count++;
                    wait_time = 0;
                }
                delay_ms_002(10);
                wait_time = wait_time + 10;
            }

            uart_print_002("Pressed ");
            uart_num_002(press_count);
            uart_print_002(" time(s)\r\n");

            for(i = 0; i < press_count; i++){
                led_on_002(LED1);
                uart_print_002("  glow ");
                uart_num_002(i + 1);
                uart_print_002("\r\n");
                delay_ms_002(300);
                led_off_002(LED1);
```

```
 48                 delay_ms_002(300);
 49             }
 50             press_count = 0;
 51             //uart_print_002("reset\r\n");
 52         }
 53     }
 54 }
~
~
"main.c" 54L, 1426B
```

Build (first time got error because function name was typed wrong, corrected and built again):

```
Build started: Project: lab1
*** Using Compiler 'V5.06 update 7 (build 960)', folder:
'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Target 1'
compiling main.c...
main.c(29): error:  #20: identifier "wait_releas_002" is undefined
main.c: 0 warnings, 1 error
".\Objects\lab1.axf" - 1 Error(s), 0 Warning(s).
Target not created.
Build Time Elapsed:  00:00:01
Build started: Project: lab1
*** Using Compiler 'V5.06 update 7 (build 960)', folder:
'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Target 1'
compiling main.c...
linking...
Program Size: Code=1236 RO-data=332 RW-data=4 ZI-data=1548
".\Objects\lab1.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:01
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 11:15:48 =~=~=~=~=~=~=~=~=~=~=~=
Q3.5 LED glows count = button press count
Pressed 1 time(s)
  glow 1
Pressed 2 time(s)
  glow 1
  glow 2
Pressed 3 time(s)
  glow 1
  glow 2
  glow 3
Pressed 1 time(s)
  glow 1
Pressed 2 time(s)
  glow 1
  glow 2
```

LED blinked the same number of times as the button was pressed.

4. Connect LEDs and control using button

Aim: Two LEDs on PA1 and PA2 and one button. 1st press LED1 on and LED2 off, 2nd press both on, 3rd press LED1 off and LED2 on. After 3rd press it starts again from 1st.

```
 1 #include "lab1_002.h"
 2
 3 // Q4 two leds controlled by one button
 4 // press 1 -> LED1 on, LED2 off
 5 // press 2 -> both on
 6 // press 3 -> LED1 off, LED2 on
 7 int press = 0;
 8
 9 int main(void){
10     gpio_init_002();
11     uart_init_002();
12     uart_print_002("Q4 two LEDs with button\r\n");
13
14     while(1){
15         if(button_read_002() == 1){
16             delay_ms_002(20);
17             if(button_read_002() == 1){
18                 press++;
```

```
 19                 if(press > 3){
 20                     press = 1;    // start again
 21                 }
 22
 23                 if(press == 1){
 24                     led_on_002(LED1);
 25                     led_off_002(LED2);
 26                     uart_print_002("press 1 : LED1 ON  LED2 OFF\r\n");
 27                 }
 28                 if(press == 2){
 29                     led_on_002(LED1);
 30                     led_on_002(LED2);
 31                     uart_print_002("press 2 : LED1 ON  LED2 ON\r\n");
 32                 }
 33                 if(press == 3){
 34                     led_off_002(LED1);
 35                     led_on_002(LED2);
 36                     uart_print_002("press 3 : LED1 OFF LED2 ON\r\n");
 37                 }
 38
 39                 while(button_read_002() == 1); // wait release
 40                 delay_ms_002(20);
 41             }
 42         }
 43     }
 44 }
~
~
"main.c" 44L, 1240B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 11:31:20 =~=~=~=~=~=~=~=~=~=~=~=
Q4 two LEDs with button
press 1 : LED1 ON  LED2 OFF
press 2 : LED1 ON  LED2 ON
press 3 : LED1 OFF LED2 ON
press 1 : LED1 ON  LED2 OFF
press 2 : LED1 ON  LED2 ON
press 3 : LED1 OFF LED2 ON
press 1 : LED1 ON  LED2 OFF
press 2 : LED1 ON  LED2 ON
```

LEDs changed on every press as given in the question.

5. Glow LED ON for 70% and OFF for 30% unit of time

Aim: LED on PA1 should be ON for 70% and OFF for 30% of the time. Taken 1 unit = 100 ms, so period = 1000 ms, ON = 700 ms, OFF = 300 ms.

Duty cycle = 700 / (700 + 300) x 100 = 70%

```
#include "lab1_002.h"

// Q5 PA1 led 70% ON, 30% OFF
// 1 unit = 100 ms, period = 10 units = 1 sec
int unit = 100;      // ms
int on_units = 7;
int off_units = 3;
unsigned int t = 0;
int cycle = 0;

int main(void){
    gpio_init_002();
    uart_init_002();
    uart_print_002("Q5 duty cycle 70/30, unit = 100 ms\r\n");

    while(1){
        cycle++;
        led_on_002(LED1);
        uart_print_002("cycle ");
        uart_num_002(cycle);
        uart_print_002(" t=");
        uart_num_002(t);
        uart_print_002(" ms ON\r\n");
        delay_ms_002(on_units * unit);
        t = t + on_units * unit;

        led_off_002(LED1);
        uart_print_002("cycle ");
        uart_num_002(cycle);
        uart_print_002(" t=");
        uart_num_002(t);
        uart_print_002(" ms OFF\r\n");
        delay_ms_002(off_units * unit);
        t = t + off_units * unit;
```

```
 35     }
 36 }
~
~
"main.c" 36L, 874B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 11:44:05 =~=~=~=~=~=~=~=~=~=~=~=
Q5 duty cycle 70/30, unit = 100 ms
cycle 1 t=0 ms ON
cycle 1 t=700 ms OFF
cycle 2 t=1000 ms ON
cycle 2 t=1700 ms OFF
cycle 3 t=2000 ms ON
cycle 3 t=2700 ms OFF
cycle 4 t=3000 ms ON
cycle 4 t=3700 ms OFF
cycle 5 t=4000 ms ON
cycle 5 t=4700 ms OFF
cycle 6 t=5000 ms ON
cycle 6 t=5700 ms OFF
cycle 7 t=6000 ms ON
cycle 7 t=6700 ms OFF
cycle 8 t=7000 ms ON
cycle 8 t=7700 ms OFF
cycle 9 t=8
```

LED was ON for 700 ms and OFF for 300 ms in every cycle.

6. When displaying the series of numbers, glow the LED only for odd numbers

Aim: Print numbers 1 to 20 on serial monitor and glow the LED on PA1 only when the number is odd (num % 2 == 1).

```
 1 #include "lab1_002.h"
 2
 3 // Q6 print numbers 1 to 20, led glows only for odd numbers
 4 int num = 1;
 5 int max_num = 20;
 6
 7 int main(void){
 8     gpio_init_002();
```

```
  9     uart_init_002();
 10     uart_print_002("Q6 LED for odd numbers\r\n");
 11
 12     while(1){
 13         uart_print_002("num = ");
 14         uart_num_002(num);
 15
 16         if(num % 2 == 1){
 17             led_on_002(LED1);
 18             uart_print_002("\todd  -> LED ON\r\n");
 19         }
 20         else{
 21             led_off_002(LED1);
 22             uart_print_002("\teven -> LED OFF\r\n");
 23         }
 24         delay_ms_002(500);
 25
 26         num++;
 27         if(num > max_num){
 28             led_off_002(LED1);
 29             uart_print_002("---- series done, restarting ----\r\n");
 30             num = 1;
 31             delay_ms_002(2000);
 32         }
 33     }
 34 }
~
~
"main.c" 34L, 767B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 11:58:40 =~=~=~=~=~=~=~=~=~=~=~=
Q6 LED for odd numbers
num = 1 odd  -> LED ON
num = 2 even -> LED OFF
num = 3 odd  -> LED ON
num = 4 even -> LED OFF
num = 5 odd  -> LED ON
num = 6 even -> LED OFF
num = 7 odd  -> LED ON
num = 8 even -> LED OFF
num = 9 odd  -> LED ON
```

```
num = 10        even -> LED OFF
num = 11        odd  -> LED ON
num = 12        even -> LED OFF
num = 13        odd  -> LED ON
num = 14        even -> LED OFF
num = 15        odd  -> LED ON
num = 16        even -> LED OFF
num = 17        odd  -> LED ON
num = 18        even -> LED OFF
num = 19        odd  -> LED ON
num = 20        even -> LED OFF
---- series done, restarting ----
num = 1 odd  -> LED ON
num = 2 even -> LED OFF
num = 3 odd  -> LED ON
```

LED glowed only for odd numbers 1, 3, 5 ... 19.

7. Modify Q5 as 40/60, 55/60, 55/55, 35/65, 90/1

Aim: Run the LED with the given ON/OFF ratios. Taken first number as ON units and second as OFF units, 1 unit = 10 ms. Each ratio runs 3 cycles.

Calculation:

40/60 : ON 400 ms, OFF 600 ms, duty = 40%

55/60 : ON 550 ms, OFF 600 ms, duty = 55/115 = 47.8%

55/55 : ON 550 ms, OFF 550 ms, duty = 50%

35/65 : ON 350 ms, OFF 650 ms, duty = 35%

90/1 : ON 900 ms, OFF 10 ms, duty = 90/91 = 98.9%

```
 1 #include "lab1_002.h"
 2
 3 // Q7 Q5 with different ON/OFF ratios
 4 // 1 unit = 10 ms
 5 int on_units[5]  = {40, 55, 55, 35, 90};
 6 int off_units[5] = {60, 60, 55, 65, 1};
 7 int unit = 10;    // ms
 8 //int unit = 100;
 9 int k, c;
10 unsigned int duty;
```

```
 11
 12 int main(void){
 13     gpio_init_002();
 14     uart_init_002();
 15     uart_print_002("Q7 different duty cycles, unit = 10 ms\r\n");
 16
 17     while(1){
 18         for(k = 0; k < 5; k++){
 19             // duty x10 to show one decimal
 20             duty = (on_units[k] * 1000) / (on_units[k] + off_units[k]);
 21
 22             uart_print_002("\r\nratio ");
 23             uart_num_002(on_units[k]);
 24             uart_print_002("/");
 25             uart_num_002(off_units[k]);
 26             uart_print_002("  ON=");
 27             uart_num_002(on_units[k] * unit);
 28             uart_print_002("ms OFF=");
 29             uart_num_002(off_units[k] * unit);
 30             uart_print_002("ms period=");
 31             uart_num_002((on_units[k] + off_units[k]) * unit);
 32             uart_print_002("ms duty=");
 33             uart_num_002(duty / 10);
 34             uart_print_002(".");
 35             uart_num_002(duty % 10);
 36             uart_print_002("%\r\n");
 37
 38             // run 3 cycles of each ratio
 39             for(c = 1; c <= 3; c++){
 40                 led_on_002(LED1);
 41                 delay_ms_002(on_units[k] * unit);
 42                 led_off_002(LED1);
 43                 delay_ms_002(off_units[k] * unit);
 44                 uart_print_002("  cycle ");
 45                 uart_num_002(c);
 46                 uart_print_002(" done\r\n");
 47             }
 48         }
 49     }
 50 }
~
~
"main.c" 50L, 1536B
```

Output:

```
=~=~=~=~=~=~=~=~=~=~=~= PuTTY log 2026.11.03 12:12:57 =~=~=~=~=~=~=~=~=~=~=~=
Q7 different duty cycles, unit = 10 ms

ratio 40/60  ON=400ms OFF=600ms period=1000ms duty=40.0%
  cycle 1 done
  cycle 2 done
  cycle 3 done

ratio 55/60  ON=550ms OFF=600ms period=1150ms duty=47.8%
  cycle 1 done
  cycle 2 done
  cycle 3 done

ratio 55/55  ON=550ms OFF=550ms period=1100ms duty=50.0%
  cycle 1 done
  cycle 2 done
  cycle 3 done

ratio 35/65  ON=350ms OFF=650ms period=1000ms duty=35.0%
  cycle 1 done
  cycle 2 done
  cycle 3 done

ratio 90/1  ON=900ms OFF=10ms period=910ms duty=98.9%
  cycle 1 done
  cycle 2 done
  cycle 3 done

ratio 40/60  ON=400ms OFF=600ms period=1000ms duty=40.0%
  cycle 1 done
```

40/60 and 35/65 the LED was off for more time. 55/55 and 55/60 looked almost same. For 90/1 the LED looked like it is always ON, the 10 ms off was not visible.

Result: All the programs were executed on STM32F103C8T6 and the output was verified.